

TorchJD: Jacobian Descent in PyTorch

Valérian Rey — valerian.rey@gmail.com
Pierre Quinton — pierre.quinton@epfl.ch

April 13, 2026



github.com/TorchJD/torchjd



torchjd.org



arxiv.org/pdf/2406.16232

Motivation

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, as

$$\begin{aligned} \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, as

$$\begin{aligned} \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

Examples:

► $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, as

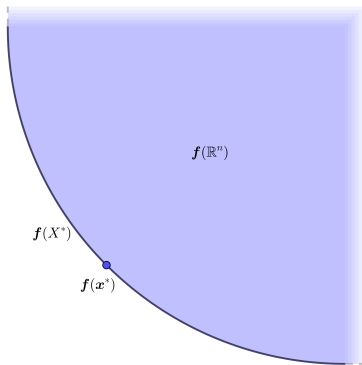
$$\begin{aligned} \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

Examples:

▶ $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

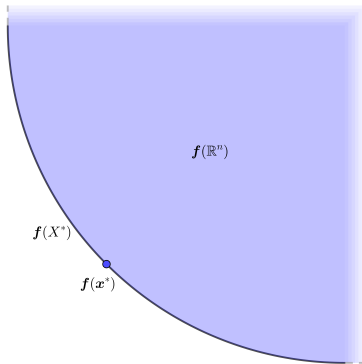
▶ $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ are not comparable with this partial order.

Pareto optimality



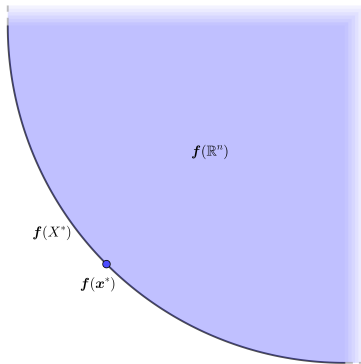
Pareto optimality

Pareto set: X^*



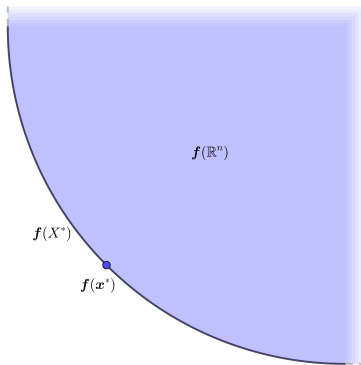
Pareto optimality

Pareto set: X^*
Pareto front: $\mathbf{f}(X^*)$



Pareto optimality

Pareto set: X^*
Pareto front: $\mathbf{f}(X^*)$



$$\mathbf{x}^* \in X^*$$

$$\Leftrightarrow$$

$$\forall \mathbf{y} \in \mathbb{R}^n, \mathbf{f}(\mathbf{y}) \preceq \mathbf{f}(\mathbf{x}^*) \rightarrow \mathbf{f}(\mathbf{y}) = \mathbf{f}(\mathbf{x}^*)$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order.

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update $\mathbf{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y}$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update $\mathbf{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y} \preceq \mathbf{f}(\mathbf{x})$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update $\mathbf{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y} \preceq \mathbf{f}(\mathbf{x})$$

We select $\mathbf{y} = -\eta \mathcal{A}(\mathcal{J}\mathbf{f}(\mathbf{x})) \in \mathbb{R}^n$.

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Algorithm 1: Jacobian descent with aggregator $\mathcal{A}$

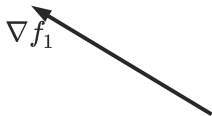
Input: $\mathbf{x} \in \mathbb{R}^n$, $0 < \eta$, $T \in \mathbb{N}$, $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$

for $t \leftarrow 1$ **to** T **do**

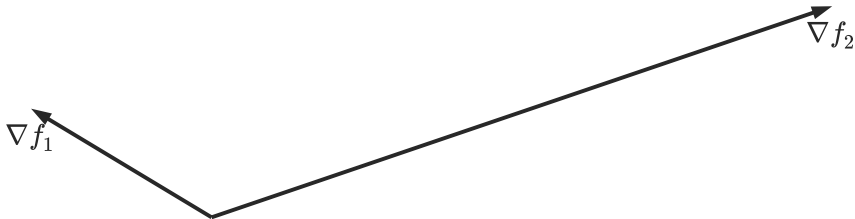
$\mathbf{x} \leftarrow \mathbf{x} - \eta \mathcal{A}(\mathcal{J}\mathbf{f}(\mathbf{x}))$

Output: $\mathbf{x}$

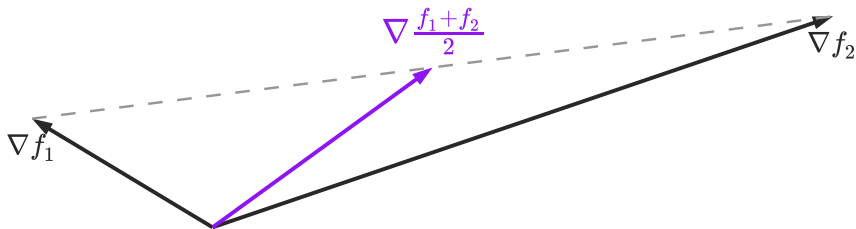
Jacobian descent



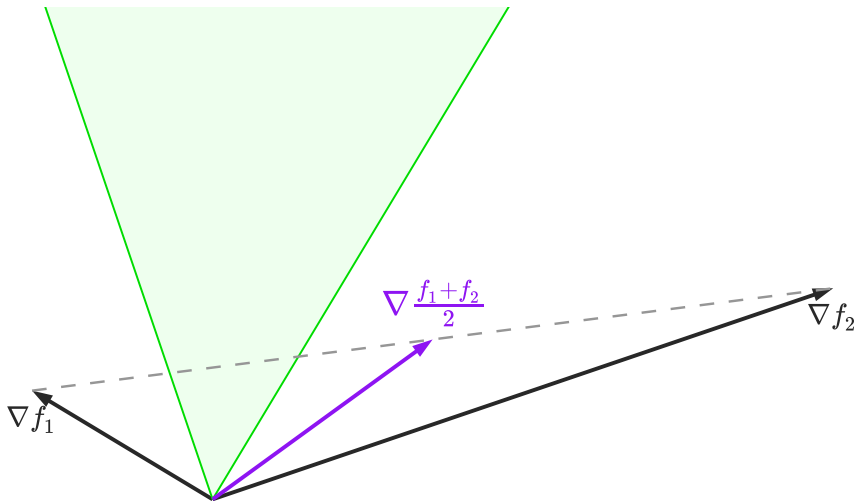
Jacobian descent



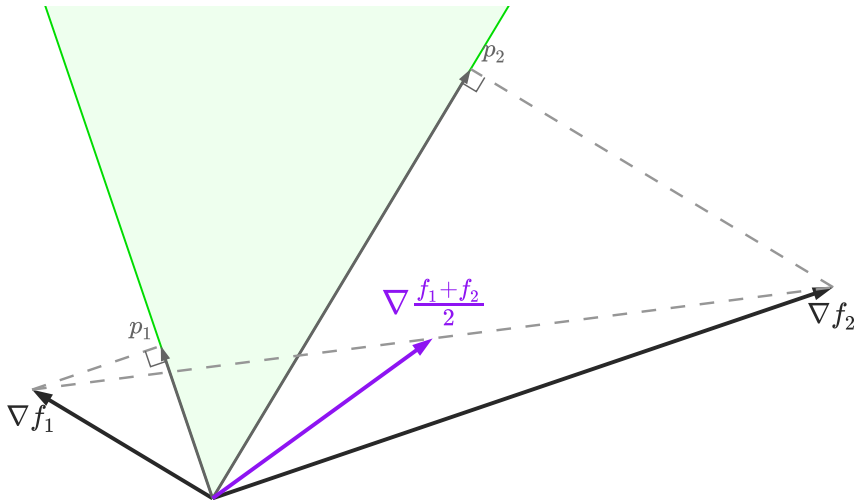
Jacobian descent



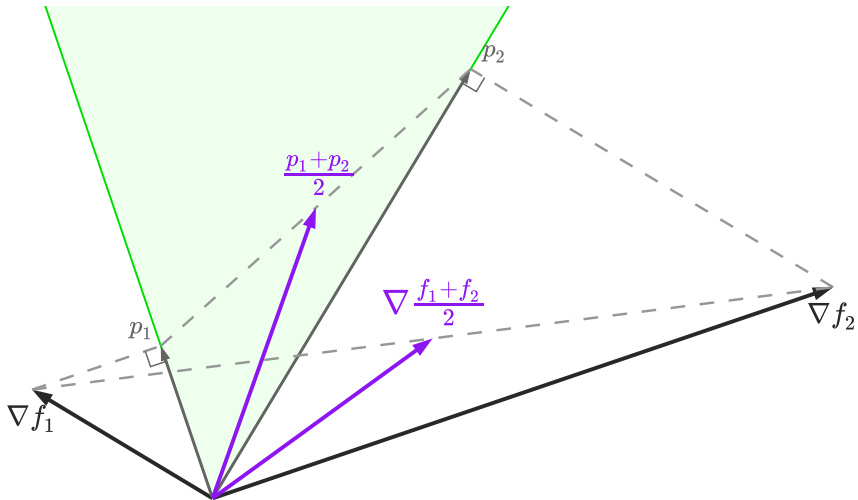
Jacobian descent



Jacobian descent



Jacobian descent

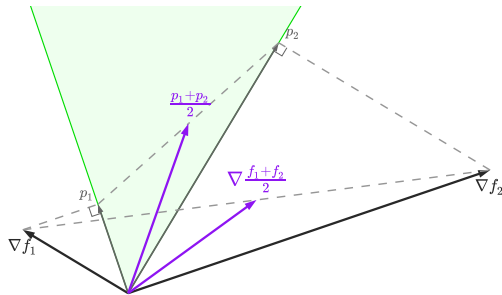


Formal definition of UPGrad

Given $J \in \mathbb{R}^{m \times n}$, let C_J be the cone $\{\mathbf{y} \in \mathbb{R}^n : \mathbf{0} \preceq J\mathbf{y}\}$.

$$\pi_J(\mathbf{x}) = \arg \min_{\mathbf{y} \in C_J} \|\mathbf{x} - \mathbf{y}\|^2$$

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m} \sum_{i \in [m]} \pi_J(\mathbf{g}_i)$$



TorchJD

TorchJD: core functions

Library	Function	Description
torch	autograd.grad	compute gradients
torch	autograd.backward	compute gradients and accumulate in .grad

TorchJD: core functions

Library	Function	Description
torch	autograd.grad	compute gradients
torch	autograd.backward	compute gradients and accumulate in .grad
torchjd	autojac.jac	compute jacobians

TorchJD: core functions

Library	Function	Description
torch	autograd.grad	compute gradients
torch	autograd.backward	compute gradients and accumulate in .grad
torchjd	autojac.jac	compute jacobians
torchjd	autojac.backward	compute jacobians and accumulate in .jac

TorchJD: core functions

Library	Function	Description
torch	autograd.grad	compute gradients
torch	autograd.backward	compute gradients and accumulate in .grad
torchjd	autojac.jac	compute jacobians
torchjd	autojac.backward	compute jacobians and accumulate in .jac
torchjd	autojac.jac_to_grad	aggregate .jac fields into .grad

TorchJD: basic usage

```
from torchjd import autojac
```

```
from torchjd.aggregation import UPGrad
```

TorchJD: basic usage

```
from torchjd import autojac
```

```
from torchjd.aggregation import UPGrad
```

```
batch_size = 16
```

```
loss_fn = MSELoss(reduction="none") # do not average over the batch
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss(reduction="none") # do not average over the batch

for input, target in dataloader:
    output = model(input) # shape: [16]
    losses = loss_fn(output, target) # shape: [16]
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss(reduction="none") # do not average over the batch

for input, target in dataloader:
    output = model(input) # shape: [16]
    losses = loss_fn(output, target) # shape: [16]
    autojac.backward(losses)
    # parameters now have a .jac field
```


TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss(reduction="none") # do not average over the batch

for input, target in dataloader:
    output = model(input) # shape: [16]
    losses = loss_fn(output, target) # shape: [16]
    autojac.backward(losses)
    # parameters now have a .jac field
    autojac.jac_to_grad(model.parameters(), UPGrad())
    # parameters now have a .grad field
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss(reduction="none") # do not average over the batch

for input, target in dataloader:
    output = model(input) # shape: [16]
    losses = loss_fn(output, target) # shape: [16]
    autojac.backward(losses)
    # parameters now have a .jac field
    autojac.jac_to_grad(model.parameters(), UPGrad())
    # parameters now have a .grad field
    optimizer.step()
    optimizer.zero_grad()
```

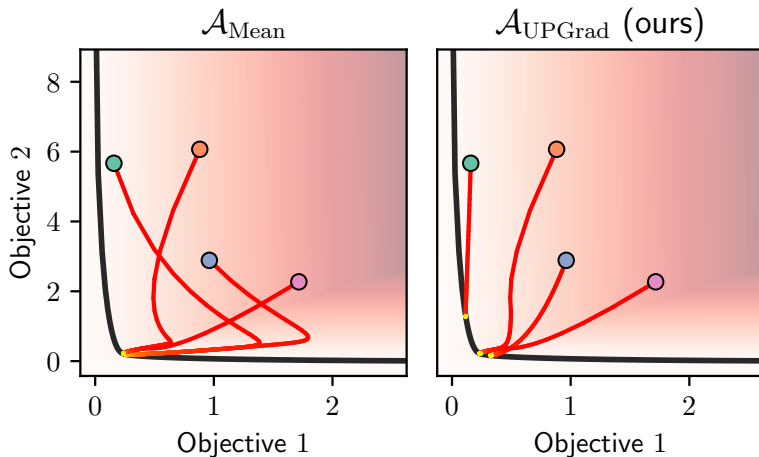
Optimization trajectories

Goal: Minimize some convex quadratic function $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, with conflicting

gradients.

Optimization trajectories

Goal: Minimize some convex quadratic function $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, with conflicting



gradients.

Discussion